Detecting Fraudulent Transactions — Final Report

Author: Johann Malto · **Date:** October 5, 2026

1. Problem understanding & framing

Financial institutions lose money and customer trust when fraudulent transactions succeed, but they also lose money and goodwill when legitimate transactions are wrongly blocked. The objective is therefore not simply to "catch fraud" but to **catch as much fraud as possible while keeping false alarms low enough that genuine customers are not disrupted.**

- **Task type:** supervised **binary classification** (fraud = 1, legitimate = 0), with severe class imbalance and a strong anomaly-detection character.
- **Primary metric: AUC-PR** (precision-recall AUC), the appropriate summary for rare-positive problems. **Accuracy is rejected** — a model predicting "legitimate" always scores ~99.8%.
- **Secondary metrics:** recall, precision, F1/F2, ROC-AUC.
- **Business KPIs:** fraud dollars prevented, false-positive rate (customer friction), investigation cost, and overall net benefit.

2. Data collection & understanding

Source: Kaggle Credit Card Fraud Detection (ULB / Worldline) — real European card transactions, September 2013.

Overview: 284,807 transactions × 31 columns; ~0.172% fraud (492 cases); no missing values. After removing 1,081 exact duplicates: 283,726 rows, 473 fraud (0.167%).

Data dictionary

Variable	Type	Description	Values
`Time`	int	Seconds since first transaction	0 – ~172,792
`V1 – V28`	float	Anonymised PCA components (confidential original features)	continuous, ~centred on 0
`Amount`	float	Transaction amount (EUR)	≥ 0
`Class`	int	Target label	0 = legit, 1 = fraud

Known limitation: `V1` – `V28` are PCA-anonymised, so features are not interpretable and domain feature engineering on them is impossible.

3. Preprocessing, EDA & feature engineering

(Notebook: `notebooks/01_eda_feature_engineering.ipynb`.)

- **Cleaning:** verified zero missing values; dropped 1,081 exact duplicate rows. Outliers were **kept** — in fraud detection the tails are often the signal.
- **Feature engineering:** derived `Hour` from `Time`, `Log_Amount = log1p(Amount)` to tame right-skew, and quantile `Amount_bin`. No categorical encoding needed (all features numeric).
- **Leakage prevention:** stratified 80/20 split **before** scaling; `RobustScaler` fit on `Time` / `Amount` / `Log_Amount` / `Hour` on the **training set only** (the `V` features are already PCA-scaled).
- **EDA findings:** the most class-separating features were **V14, V12, V4, and V10**; in the t-SNE projection fraudulent transactions form distinct, well-separated clusters, confirming the classes are separable in feature space.
- **Feature selection:** mutual information (filter), tree importances (embedded), and RFE (wrapper); features strong across methods: V14, V12, V11, V10, V4, plus an amount-based feature (Amount_bin / Log_Amount).
- **Dimensionality reduction:** PCA needs 23 components for 95% variance (data is already PCA'd, so this is largely a formality); t-SNE used for 2-D visualisation only.

4. Model implementation

(Notebook: `notebooks/02_model_implementation.ipynb`; script: `src/train.py`.)

- **Imbalance handling:** balanced class weights / `scale_pos_weight`; decision threshold tuned for **F2** on training cross-validation (no test leakage).
- **Supervised models:** Logistic Regression, Decision Tree, Random Forest, XGBoost.
- **Unsupervised baseline:** Isolation Forest.
- **SMOTE check:** a RandomForest + SMOTE pipeline (oversampling inside CV folds only) was tested and scored *slightly below* plain class weighting (AUC-PR 0.789 vs 0.793), so class weighting was retained.
- **Scoping:** SVM excluded (does not scale to ~280k rows); deep learning omitted (trees dominate tabular data); recommendation task not applicable.

5. Results

(Source: `models/metrics_comparison.csv`, notebook `02`. Metrics are on the held-out test set at each model's F2-tuned threshold.)

Model	AUC-PR	ROC-AUC	Recall	Precision	F2
XGBoost	**0.817**	0.980	0.779	0.914	0.803
Random Forest	0.793	0.966	0.779	0.841	0.791
Logistic Regression	0.683	0.968	0.768	0.839	0.782
Decision Tree	0.497	0.907	0.000	0.000	0.000
Isolation Forest	0.111	0.944	0.253	0.253	0.253

The Decision Tree's F2-tuned threshold collapsed to predicting no fraud at test (hence 0 precision/recall); its AUC-PR of 0.497 still reflects ranking ability. The random baseline AUC-PR is ~0.0017, so every supervised model beats chance by orders of magnitude.

Champion: XGBoost (AUC-PR 0.817). On the test set at the F2-tuned threshold it caught **74 of 95 frauds** (recall 0.78) with only **7 false alarms** (precision 0.91). Supervised models clearly outperform the unsupervised baseline (Isolation Forest, AUC-PR 0.11), showing that labels add substantial value on this problem.

6. Bias & fairness analysis

See the dedicated section in `Bias_Fairness_Analysis.md`. Key points: the dataset has **no sensitive attributes**, so a true demographic audit is impossible (an ethical finding in itself); we demonstrate the methodology on a declared `Amount` proxy and find no major disparity (disparate-impact ratio 0.863 — PASS; ΔFPR ≈ 0.000; a small ΔTPR of 0.062 worth monitoring). Model decisions are explained with feature importances, SHAP, PDP, ICE, and LIME, which agree on V14, V12, V4, and V10 as the dominant drivers.

The explainability and fairness audit were run on the **Random Forest** as a representative tree model (a close runner-up at AUC-PR 0.793); repeating them on the XGBoost champion is a recommended next step, though the two tree ensembles are expected to behave similarly.

7. Limitations

- Anonymised features limit interpretability.
- No demographic data for a genuine fairness audit.
- Two days of 2013 data may not generalise to current fraud patterns.
- **Overfitting:** quantified on the Random Forest (the model used for interpretability) via the train-vs-test AUC-PR gap: **0.980 vs 0.793, a gap of 0.187**. This is a sizeable gap, indicating some overfitting — expected for a depth-12 Random Forest on highly imbalanced data; stronger regularisation (shallower trees, higher `min_samples_leaf`) or more data would narrow it. The XGBoost champion reaches a higher test AUC-PR (0.817); a train-vs-test check on

the champion is a recommended next step.

8. Conclusion & next steps

The **XGBoost** champion achieves **0.817** AUC-PR — catching 74 of 95 test frauds with only 7 false alarms — at a controlled false-alarm rate suitable for a real review workflow. The explainability analysis shows the model relies on a small, stable set of genuine signals (V14, V12, V4, V10), and the fairness audit surfaces no major disparity on the available proxy. Recommended next steps: deploy in shadow mode with a dollar-based success threshold; repeat the explainability and fairness audit on the XGBoost champion; apply stronger regularisation to narrow the train/test gap; add lawfully consented features to enable a real demographic fairness audit; and monitor for drift in production.

9. Use of Generative AI

Generative AI was used as a supporting tool throughout this project. Specifically, it assisted with: writing and debugging code for the data preprocessing, modelling, and analysis notebooks; drafting and formatting the written deliverables, including this report and the bias & fairness analysis; and advising on methodology and best practices — suggesting improvements to strengthen the capstone, such as the choice of AUC-PR over accuracy, leakage-prevention steps, and how to frame the fairness audit.

All AI-assisted outputs were reviewed, executed, and validated by the author, who made the final decisions on methodology, interpretation, and conclusions. The results and findings reported here reflect the author’s own verified work.